



OBJECT ORIENTED SYSTEMS (OOS) — 2023

QUESTION 4 COMPLETE DETAILED SOLUTION



Introduction

Question 4 of the 2023 Object Oriented Systems examination belongs to CO4 and focuses on one of the most essential areas of Object-Oriented Software Engineering:



Unified Modeling Language (UML)

UML is a standardized visual modeling language used to:

- analyze software systems,
- design object-oriented architecture,
- visualize interactions,
- and document software structure.

Before implementing large software systems, software engineers use UML diagrams to understand object interactions, relationships, responsibilities, dependencies, and system behavior. Without UML modeling, large software systems often become difficult to manage, poorly structured, and hard to maintain ⚠️.

This question mainly discusses:

- **Use Case Diagrams**
- **Class Diagrams**
- **Association Relationships**
- **Aggregation**
- **Composition**
- **Multiplicity**
- **Inheritance Representation**

These concepts form the foundation of software architecture and object-oriented system design 🚀☕.



QUESTION 4(a)



Explain Use Case Diagram in UML with suitable examples.



Answer

A **Use Case Diagram** is one of the most important behavioral diagrams in UML. It represents the functional requirements of a software system from the user's perspective.

In simple words, a use case diagram describes *what* the system does, not *how* the system works internally. It captures interactions between users and the system, system functionalities, and external behavior. Thus, use case diagrams help software developers understand user expectations, system requirements, and application functionality clearly 🚀.



Why Use Case Diagrams are Needed?

During software development, understanding user requirements is extremely important. If requirements are misunderstood, the wrong software may be developed, functionality may become incomplete, and project failure may occur ⚠️.

Use case diagrams solve this problem by visually representing system functionalities, user interactions, and external operations. Thus, communication between clients, analysts, designers, and developers becomes easier.

🌟 Components of Use Case Diagram

Component	Purpose
Actor	External entity interacting with system
Use Case	Functional behavior
System Boundary	Defines system scope
Relationship	Interaction between components

🌟 Actor

An **Actor** represents a user, external system, device, or application that interacts with the software system. Actors are placed outside the system boundary. *Examples:* Customer, Admin, Student, ATM User.

🌟 Use Case

A **Use Case** represents functionality, a service, or an operation performed by the system. Use cases are represented using oval shapes. *Examples:* Login, Withdraw Money, Book Ticket, Pay Fees.

🌟 Example of Use Case Diagram

Consider an ATM system.

- **Actors:** Customer
- **Use Cases:** Withdraw Money, Check Balance, Deposit Money

Representation:

```
Customer ---- (Withdraw Money)
Customer ---- (Check Balance)
Customer ---- (Deposit Money)
```

Deep Explanation: Here, the Customer interacts with the ATM system. The system provides multiple services, and each service becomes a separate use case. Thus, system functionality becomes visually understandable.

🌟 Include and Extend Relationship

Use case diagrams also support **include** and **extend** relationships.

- ♦ **Include Relationship** Include represents mandatory reusable functionality. *Example:*

```
(Transfer Money) ..<<include>>.. (Authentication)
```

Authentication always occurs.

◆ **Extend Relationship** Extend represents optional behavior. *Example:*

```
(Checkout) <..<<extend>>.. (Apply Coupon)
```

Coupon application is conditional.

☀ Advantages & Real-World Applications

Use case diagrams simplify requirement analysis, improve communication between technical and non-technical stakeholders, help identify system functionality clearly, and support modular software planning. They are widely used in banking systems, hospital management, e-commerce systems, railway reservation systems, and ERP applications.

🎯 Conclusion

Use case diagrams visually represent system functionality from the user's perspective and help software engineers understand system requirements clearly and effectively 🚀.

📄 QUESTION 4(b)

? **Explain Class Diagram in UML.**

? **Discuss how relationships are represented in class diagrams.**

✅ **Answer**

A **Class Diagram** is one of the most important structural UML diagrams used to represent the static structure of an object-oriented system. Class diagrams show classes, attributes, methods, relationships, and dependencies. They act as the blueprint of object-oriented software architecture 🧠.

Before coding large systems, software engineers design class diagrams to understand object responsibilities, interactions, and system structure.

☀ Structure of a Class Diagram

A UML class is represented using a rectangle divided into three compartments. These compartments contain:

1. Class Name
2. Attributes
3. Methods

☀ Example of Class Representation

Suppose we have a Java class:

```
class Student {
    int roll;
    String name;
    void display() { }
}
```

UML Representation:

```

-----
|           Student           |
-----
| - roll : int                |
| - name : String             |
-----
| + display() : void          |
-----

```

☀ Meaning of Symbols (Access Specifiers)

Symbol	Meaning
+	Public
-	Private
#	Protected

☀ Attributes and Methods in Class Diagram

- **Attributes** represent instance variables, object data, and state information. (e.g., - roll : int means a private variable of datatype int).
- **Methods** represent object behavior, functionality, and operations. (e.g., + display() : void means a public method that returns void).

☀ Relationships in Class Diagrams

Class diagrams support several relationships: Association, Aggregation, Composition, Inheritance, and Dependency.

♦ **Association Relationship** Association represents a general relationship between objects. *Example:* Teacher teaches Student.

```
Teacher ----- Student
```

- ♦ **Multiplicity** Multiplicity specifies the number of objects participating in a relationship.

Multiplicity	Meaning
1	Exactly one
*	Many
0..1	Optional
1..*	One or more

Example: Department 1 ----- * Student (Meaning: one department, many students).

◆ **Aggregation Relationship** Aggregation represents weak ownership (HAS-A relationship). Child objects can exist independently. *Example:* Department and Teacher. Representation uses a hollow diamond ◇.

Department ◇----- Teacher

◆ **Composition Relationship** Composition represents strong ownership (PART-OF relationship). Child objects cannot exist independently. *Example:* House and Room. Representation uses a filled diamond ◆.

House ◆----- Room

◆ **Inheritance Relationship** Inheritance represents an IS-A relationship. *Example:* Dog IS-A Animal. Representation uses a hollow triangular arrow.

Animal
▲
|
Dog

🎯 Conclusion

Class diagrams represent the static architecture of object-oriented systems using classes, attributes, methods, and relationships. They provide the structural foundation of UML-based software design 🚀.

📄 QUESTION 4(c)

? **Distinguish between aggregation and composition with suitable examples.**

✅ Answer

Aggregation and composition are two very important relationships in UML class diagrams. Both represent ownership, containment, and HAS-A relationships. However, their dependency strength differs significantly 🗨️.

🌟 Aggregation

Aggregation represents a **weak ownership** relationship. In aggregation, child objects can exist independently of the parent object. Destroying the parent does NOT destroy the child. It represents loose coupling.

Example: University and Professor. Professors may exist independently even if the university closes.

Representation:

University ◇----- Professor

☀ Composition

Composition represents a **strong ownership** relationship. In composition, child objects cannot exist independently. Destroying the parent destroys the child automatically.

Example: House and Room. Rooms do not meaningfully exist independently without a house.

Representation:

House ◆----- Room

☀ Deep Conceptual Difference

Aggregation indicates *shared ownership*. Composition indicates *exclusive ownership* and lifecycle dependency. This difference becomes extremely important in database modeling, memory management, and object lifecycle control.

☀ Comparison Table

Feature	Aggregation	Composition
Ownership	Weak	Strong
Child Dependency	Independent	Dependent
Lifetime	Separate	Parent-controlled
UML Symbol	◇ (Hollow Diamond)	◆ (Filled Diamond)
Relationship Type	HAS-A	PART-OF

☀ Real-World Intuition

- Team and Player → **Aggregation** (Players may leave the team independently).
- Human and Heart → **Composition** (The heart cannot meaningfully exist independently inside the human-body model 💡).

🎯 Conclusion

Aggregation and composition both represent object ownership, but aggregation models weak independent relationships whereas composition models strong lifecycle dependency 🚀.

QUESTION 4(d)

? Explain inheritance representation in UML class diagrams with suitable examples.

✓ Answer

Inheritance is one of the most fundamental concepts of Object-Oriented Programming. It allows one class to acquire the properties, methods, and behavior of another class. Inheritance promotes code reuse, hierarchical classification, extensibility, and runtime polymorphism 🚀.

In UML class diagrams, inheritance is represented visually using special notation.

🌟 UML Representation of Inheritance

Inheritance is represented using a solid line with a hollow triangular arrow. The arrow points toward the **parent class**.

Example Representation:



Meaning: Dog inherits Animal.

🌟 Java Example

```
class Animal {  
    void sound() {  
        System.out.println("Animal Sound");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog Barking");  
    }  
}
```

🌟 Deep Explanation

Here, the **Dog** class inherits **Animal**. The **Dog** acquires the **sound()** method and parent properties. Thus, code reuse occurs automatically.

🌟 Types of Inheritance

- **Java supports:** Single inheritance, Multilevel inheritance, Hierarchical inheritance.
- **Java does NOT support:** Multiple inheritance using classes.

🌟 Advantages of Inheritance

Inheritance provides code reusability, extensibility, reduced redundancy, hierarchical classification, and runtime polymorphism.

Real-World Analogy: **Vehicle** → **Car**. A Car automatically inherits speed, engine behavior, and movement capability from a Vehicle.

🎯 Conclusion

Inheritance representation in UML class diagrams visually models hierarchical object-oriented relationships and promotes reusable software architecture 🚀.